

Full Marks:50

SET:A

Time: 1Hour 10Minutes

Q1. Suppose the current directory contains a file named random.py. What potential problem can occur with:

```
import random
```

- A)Python automatically renames the local file
- B)random cannot be imported
- C)Python will always reject it
- D)The local file can shadow the intended standard-library module

Q2. What will be printed?

```
class InvalidAgeError(Exception):
    pass
```

```
def check_age(age):
    if age < 18:
        raise InvalidAgeError("Age is
too low")
    return "Allowed"
```

```
try:
    print(check_age(15))
except InvalidAgeError as e:
    print(type(e).__name__)
    print(e)
```

- A)InvalidAgeError
- 15
- B)InvalidAgeError
- Age is too low
- C)Nothing, because custom exceptions cannot be caught
- D)Exception
- Age is too low

Q3. Suppose module_a.py contains:

```
print("Module loaded")
```

```
value = 10
```

And another file contains:

```
import module_a
import module_a
```

```
print(module_a.value)
```

Assuming both imports occur in the same Python process, what is normally printed?

- A)ImportError
- B)10
- 10
- C)Module loaded
- 10
- D)Module loaded
- Module loaded
- 10

Q4. What happens here?

```
class MyError(Exception):
    pass
```

```
def test():
    raise MyError("Something went
wrong")
```

```
try:
    test()
except MyError as e:
    print(type(e).__name__)
```

- A)MyError
- B)Exception
- C)Something went wrong
- D)TypeError

Q5. What will happen?

```
def fun(a, b=10):
    return a + b
```



```
print(fun(b=5, a=20))
```

- A)15
- B)25
- C)SyntaxError
- D)TypeError

Q6. What is the output?

```
numbers = [1, 2, 3, 4, 5]
```

```
result = list(
    map(
        lambda x: x * 2,
        filter(lambda x: x % 2 == 0,
numbers)
    )
)
```

```
print(result)
```

- A)[2, 4, 6, 8, 10]
- B)[4, 8]
- C)[1, 3, 5]
- D)[2, 4]

Q7. What is the output?

```
try:
    raise ValueError
except ValueError:
    print("A")
    raise
```

- A)B
- B)A, and execution ends normally
- C)No Output
- D)A, followed by propagation of the same exception

Q8. What does the following function return?

```
def fun():
    print("Hello")
```

```
x = fun()
```

```
print(x)
```

- A)None
- B>Error
- C>Hello None
- D>Hello

Q9. Suppose sample.py contains:

```
x = 10
```

```
def display():
    print("Hello")
```

What will this print?

```
import sample
```

```
print(sample.x)
sample.display()
```

- A)x
- display
- B)10
- Hello
- C>Error
- D)Only Hello

Q10. What will be printed?

```
def fun(a, b):
    return a + b
```

```
print(fun(10, b=20))
```

- A)TypeError
- B)20
- C)SyntaxError
- D)30

Q11. What happens when this code executes?

```
def fun():
    print(variable)
    variable = 50
```

```
variable = 10
```

```
fun()
```

- A)UnboundLocalError
- B)NameError
- C)10
- D)50

Q12. What will be printed?

```
from functools import reduce
```

```
numbers = [1, 2, 3, 4]
```



```
result = reduce(
    lambda a, b: a * b,
    numbers
)
```

```
print(result)
```

A)24

B)0

C)9

D)10

Q13. What happens here?

```
def fun(a):
    return a
```

```
print(fun())
```

A)TypeError

B)None

C)0

D)NameError

Q14. What will be printed?

```
try:
    print(10 / 0)
except ZeroDivisionError:
    print("A")
except Exception:
    print("B")
```

A)A B

B)A

C)B

D)Nothing

Q15. Suppose test.py contains:

```
print("A")
```

```
if __name__ == "__main__":
    print("B")
```

If another Python program executes:

```
import test
```

what will normally be printed?

A)B

B)A and B

C)Nothing

D)A

Q16. What will be the output?

```
x = 10
```

```
def outer():
    x = 20
```

```
def inner():
    global x
    x = 30
```

```
inner()
print(x)
```

```
outer()
print(x)
```

A)30

30

B)20

30

C)20

10

D)30

20

Q17. What is the output?

```
try:
    print(10 / 0)
except Exception:
    print("A")
except ZeroDivisionError:
    print("B")
```

A)B

B)A B

C)A

D)SyntaxError

Q18. What happens when this code runs?

```
def first():
    second()
```

```
def second():
    third()
```

```
def third():
    raise RuntimeError("Failure")
```



```
try:
    first()
except RuntimeError:
    print("Handled")
A)Handled
B)TypeError
C)NameError
D)RuntimeError cannot propagate through functions
```

Q19. What is the major difference between the following?

```
import math
```

```
and
```

```
from math import sqrt
```

- A)Both import only sqrt
- B)First imports only sqrt
- C)There is no difference
- D)First provides the module namespace; second imports sqrt directly

Q20. What will be printed?

```
def make_counter():
    count = 0
```

```
    def counter():
        nonlocal count
        count += 1
        return count
```

```
    return counter
```

```
a = make_counter()
b = make_counter()
```

```
print(a())
print(a())
print(b())
print(a())
```

- A)1
- 2
- 1
- 3
- B)1
- 1
- 1

- 1
- C)1
- 2
- 3
- 4
- D)0
- 1
- 0
- 2

Q21. What will be the output?

```
x = 10
```

```
def fun():
    print(x)
    x = 20
```

```
fun()
```

- A)10
- B)UnboundLocalError
- C)20
- D)NameError

Q22. What will be printed?

```
def fun():
    print("A")
    return
    print("B")
```

```
fun()
```

- A)A
- B)B
- C)Nothing
- D)A B

Q23. What will be printed?

```
def fun(x=[]):
    x.append(10)
    return x
```

```
a = fun()
b = fun()
```

```
print(a)
print(b)
A>Error
B)[10]
[10]
```



C)[10, 10]

[10]

D)[10]

[10, 10]

Q24. What will be the output?

```
import math
```

```
print(math.sqrt(25))
```

A)5

B)25

C)5.0

D)sqrt(25)

Q25. What is the output?

```
f = lambda x: x + 5
```

```
print(f(10))
```

A)5

B)10

C>Error

D)15

Q26. Why is

```
try:
```

```
    print(10 / 0)
```

```
except Exception:
```

```
    print("A")
```

```
except ZeroDivisionError:
```

```
    print("B")
```

codes's second handler never reached?

A)except blocks execute randomly

B)ZeroDivisionError occurs before Exception

C)ZeroDivisionError is not an exception

D)Exception is a superclass of ZeroDivisionError

Q27. What will be printed?

```
x = 10
```

```
def outer():
```

```
    x = 20
```

```
    def inner():
```

```
        print(x)
```

```
    x = 30
```

```
inner()
```

```
outer()
```

A)30

B)10

C)NameError

D)20

Q28. What is the output?

```
def fun():
```

```
    try:
```

```
        print("A")
```

```
        raise ValueError
```

```
    except ValueError:
```

```
        print("B")
```

```
        return 10
```

```
    else:
```

```
        print("C")
```

```
        return 20
```

```
    finally:
```

```
        print("D")
```

```
print(fun())
```

A)A

B

C

D

10

B)A

C

D

20

C)A

B

10

D

D)A

B

D

10

Q29. What is the output?

```
def outer():
```

```
    x = 10
```

```
    def inner():
```

```
        return x + 5
```



return inner

**f = outer()
print(f())**

- A)15
- B)None
- C)10
- D)NameError

Q30. What is the output?

```
def fun():  
    try:  
        return 10  
    finally:  
        return 20
```

print(fun())

- A)20
- B)10
- C)RuntimeError
- D)None

Q31. What happens here?

from math import sqrt

print(sqrt(36))

- A)6
- B)6.0
- C)NameError
- D)36

Q32. What will be the output?

```
def fun(x):  
    try:  
        if x < 0:  
            raise ValueError("negative")  
        return x  
    except ValueError:  
        return 100  
    finally:  
        print("cleanup")
```

print(fun(-5))

- A)cleanup
100
- B)negative
cleanup

- C)100
cleanup
- D)cleanup
-5

Q33. What will be printed?

```
def fun():  
    try:  
        return 10  
    finally:  
        print("Finally")
```

print(fun())

- A)10 only
- B)Finally only
- C)10 followed by Finally
- D)Finally followed by 10

Q34. What will happen?

```
def fun(n):  
    if n == 0:  
        return 1  
    return n * fun(n)
```

print(fun(3))

- A)TypeError
- B)3
- C)RecursionError
- D)6

Q35. What will be printed?

x = 10

f = lambda: x

x = 20

print(f())

- A)NameError
- B)20
- C)None
- D)10

Q36. What is the output?

```
def fun(x):  
    if x == 0:  
        return  
    return x + fun(x - 1)
```



print(fun(3))

- A)None
- B)6
- C)3
- D)5

Q37. What is the output?

```
def fun(x):  
    return x * 2
```

```
a = fun  
print(a(5))
```

- A)Error
- B)10
- C)func(5)
- D)5

Q38. What will happen?

```
def outer():  
    x = 10  
  
    def inner():  
        nonlocal y  
        y = 20
```

```
    inner()
```

```
outer()  
A)NameError  
B)SyntaxError  
C)20  
D)10
```

Q39. What is the output?

```
def fun(a=10, b=20):  
    return a - b
```

```
print(fun(30))  
A)20  
B)30  
C)10  
D)-10
```

Q40. What is the purpose of this statement?

```
if __name__ == "__main__":  
A)It prevents syntax errors
```

- B)It creates the main function
- C)It identifies code intended to run when the file is executed directly
- D)It prevents all imports

Q41. What happens here?

```
try:  
    raise ValueError("Wrong value")  
except ValueError as e:  
    print(e)
```

- A)ValueError
- B)Nothing
- C)Wrong Value
- D)e

Q42. What happens when this function is called?

```
def fun(a=10, b):  
    return a + b
```

- A)SyntaxError
- B)It executes correctly
- C)NameError
- D)TypeError

Q43. What will be printed?

```
try:  
    x = int("10")  
except ValueError:  
    print("A")  
else:  
    print("B")  
finally:  
    print("C")
```

- A)B C
- B)B
- C)A C
- D)A

Q44. What is the output?

```
def fun(x):  
    x += 10  
    return x
```

```
x = 5  
y = fun(x)
```

```
print(x, y)  
A)15 5
```



B)15 15

C)5 5

D)5 15

Q45. What is the output?

```
def outer():
```

```
    x = 10
```

```
    def inner():
```

```
        nonlocal x
```

```
        x = x + 5
```

```
        return x
```

```
    inner()
```

```
    return x
```

```
print(outer())
```

A)10

B)UnboundLocalError

C)5

D)15

Q46. Which object keeps track of modules that have already been imported during the current Python process?

A)sys.imports

B)sys.path

C)__module__

D)sys.modules

Q47. What happens?

```
def fun(a, b):
```

```
    return a + b
```

```
print(fun(10, a=20))
```

A)20

B)TypeError

C)30

D)SyntaxError

Q48. What is the output?

```
try:
```

```
    x = int("abc")
```

```
except ValueError:
```

```
    print("A")
```

```
else:
```

```
    print("B")
```

```
finally:
```

```
print("C")
```

A)A

B)B

C)B C

D)A C

Q49. What will normally happen?

```
import math
```

```
import math
```

A)The module is normally loaded once and reused

B)The module is normally loaded and executed twice

C)ImportError

D)SyntaxError

Q50. if test.py has

```
print("A")
```

```
if __name__ == "__main__":
```

```
    print("B")
```

If test.py is executed directly, what will be printed?

A)A

B)A followed by B

C)Nothing

D)B

